MSc Computer Science — Distributed Computing

Lab 01

Distributed Client-Server Architecture

Go Language | 5-Node LAN Cluster | 1 Hour

Your Group Details

Group Number			
_____			

Learning Objectives

By the end of this lab you will be able to:

- Set up real TCP communication across a 5-node LAN cluster in Go
- Implement concurrent client handling using goroutines
- Handle network timeouts, connection limits, and graceful shutdown
- Observe and explain what happens when a distributed system fails
- Measure and interpret latency and throughput under different conditions

📋 **Before This Lab**

You should have completed the Go self-study exercises (go_exercises.docx) and the Basics and Concurrency sections of A Tour of Go at go.dev/tour. If you have not, review the Go Quick Reference on the last page of this sheet before starting.

Cluster Overview

Node	Machine IP	Role	Your container
Node 1	10.8.100.22	SERVER	lab01-gX-server
Node 2	10.8.100.1	Client	lab01-gX-client2
Node 3	10.8.100.2	Client	lab01-gX-client3
Node 4	10.8.100.12	Client	lab01-gX-client4
Node 5	10.8.100.28	Client	lab01-gX-client5

Replace X with your group number in all container names. Example for Group 3: lab01-g3-server

Getting Connected

Step 1 — SSH Into Your Assigned Machine

Open a terminal and SSH in using your group credentials:

```
# Replace X with your group number
# Node 1 (Server machine)
ssh groupX@10.8.100.22

# Node 2
ssh groupX@10.8.100.1

# Node 3
ssh groupX@10.8.100.2

# Node 4
ssh groupX@10.8.100.12

# Node 5
ssh groupX@10.8.100.28

# Password: groupX@123   (e.g. group3@123 for Group 3)
```

Step 2 — Enter Your Container

Once logged into the machine, enter your group's container:

```
# Replace X with your group number

# Node 1 — Server container
docker exec -it lab01-gX-server bash

# Node 2
docker exec -it lab01-gX-client2 bash

# Node 3
docker exec -it lab01-gX-client3 bash

# Node 4
docker exec -it lab01-gX-client4 bash

# Node 5
docker exec -it lab01-gX-client5 bash
```

You should see a prompt like:

```
root@hostname:/lab01/server#
```

Your lab files are at:

```
/lab01/server/server.go    ← server skeleton code (Node 1)
/lab01/client/client.go    ← client skeleton code (Nodes 2–5)
/lab01/server/server_bin   ← pre-compiled server binary
/lab01/client/client_bin   ← pre-compiled client binary
```

Step 3 — Node 1 Only: Verify Server is Running

> ⚠️ **Important**
> The server starts AUTOMATICALLY when the container launches. Do NOT run ./server_bin manually — the port is already in use and you will get an error.

Verify the server is running:

```
ps aux | grep server_bin
# You should see: 7  ./server_bin -port 7000
```

Step 4 — Nodes 2–5: Test the Client

Run the client to confirm you can reach the server:

```
cd /lab01/client
./client_bin -server 10.8.100.22:7000 -workers 3 -requests 30
# Replace 7000 with YOUR group's port

# Expected output (before completing TODOs):
[CLIENT] Connecting to server: 10.8.100.22:7000
[CLIENT] Workers: 3 | Requests: 30 | Delay: 100ms
========== LAB 01 RESULTS ==========
Total time:     0.00s
Successes:      0
Failures:       0
=====================================

# 0 successes is CORRECT — the TODOs are not filled in yet
```

> 💡 **Why 0 Successes?**
> The skeleton code has empty TODO functions. The client connects but sends nothing. As you complete each task, you will see the results improve. That is the point of the lab!

Coding Tasks

Open the skeleton files and find each TODO comment. Complete all 6 tasks. Use nano to edit:

```
nano /lab01/server/server.go    # for Tasks 1, 2, 3
nano /lab01/client/client.go    # for Tasks 4, 5, 6
```

Task 1 server.go	**`acceptLoop()`** [Medium]	Reject new connections when the server is at maximum capacity
Task 2 server.go	**`handleConnection()`** [Medium]	Read requests in a loop, simulate work with a delay, send responses back
Task 3 server.go	**`waitForShutdown()`** [Hard]	Wait for all active connections to finish before the server exits (10 second timeout)
Task 4 client.go	**`runWorker()`** [Medium]	Establish a TCP connection to the server with timeout and error handling
Task 5 client.go	**`sendRequest()`** [Hard]	Send a request message and read the response — set deadlines on both
Task 6 client.go	**`collectResults()`** [Easy]	Print details of any failures and the final summary statistics

How to Recompile After Editing

> ⚠️ **Always Kill the Running Server First**
> If you are on Node 1 and want to recompile server.go, you must kill the running server first — otherwise the new binary cannot bind to the port.

```
# Node 1 – recompile server
pkill server_bin              # kill running server first
cd /lab01/server
go build -o server_bin server.go
./server_bin -port 7000       # replace 7000 with your port

# Nodes 2–5 – recompile client
cd /lab01/client
go build -o client_bin client.go
./client_bin -server 10.8.100.22:7000 -workers 3 -requests 30
```

Expected Output After Completing All Tasks

Once all 6 TODOs are implemented, running the client should produce:

```
[CLIENT] Connecting to server: 10.8.100.22:7000
[CLIENT] Workers: 3 | Requests: 30 | Delay: 100ms
[WORKER 0] Connected to server
[WORKER 1] Connected to server
[WORKER 2] Connected to server
...
========== LAB 01 RESULTS ==========
Total time:     5.23s
Successes:      30
Failures:       0
Avg latency:    52ms
Throughput:     5.7 req/s
Success rate:   100.0%
====================================
```

Experiments

Complete all 6 coding tasks first. Then run each experiment in order. Record results in the table at the end.

Experiment A — Baseline — Everything Working

Steps:

1. Make sure all 5 nodes are connected and server is running
2. Run the client on each of Nodes 2, 3, 4, 5 simultaneously
3. Let all clients complete their requests
4. Record average latency and throughput from the results output

Observe & Record:

→ What is the average request latency?
→ What is the throughput (requests per second)?
→ Do all 4 client nodes get similar latency?

Experiment B — Overload — Too Many Concurrent Clients

Steps:

5. On Node 1, restart server with a low limit: pkill server_bin then ./server_bin -port 7000 -maxconn 5
6. On all 4 client nodes simultaneously run: ./client_bin -server 10.8.100.22:7000 -workers 20 -requests 100
7. Watch both server stats and client output

Observe & Record:

→ How many connections does the server reject?
→ What error message do rejected clients see?
→ Does latency change for accepted connections?

Experiment C — Server Crash — Mid-Request Failure

Steps:

8. Start all clients running normally with default settings
9. While clients are actively sending, press Ctrl+C on the Node 1 server terminal
10. Watch the client terminals immediately after

Observe & Record:

→ How quickly do clients detect the crash?
→ What error appears? (connection refused / timeout / reset by peer)
→ Do some clients detect the failure faster? Why?

Experiment D — Network Partition — iptables

Steps:

11. Start server and all clients running normally
12. On Node 1 inside the container, block traffic from Node 2:
13. sudo iptables -A INPUT -s 10.8.100.1 -j DROP
14. Watch Node 2 client output for 30 seconds

15. Restore: sudo iptables -D INPUT -s 10.8.100.1 -j DROP
16. Repeat using REJECT instead of DROP — compare the difference

Observe & Record:

→ With DROP — how long before Node 2 times out? Why does it take so long?
→ With REJECT — does the error appear faster? Why is REJECT different?
→ After restoring, does the client recover automatically?

Experiment E — Network Latency — Artificial Delay

Steps:

17. On Node 1 inside the container, add 500ms artificial latency:
18. sudo tc qdisc add dev eth0 root netem delay 500ms
19. Run clients and observe latency in results
20. Try 200ms and 1000ms — record each
21. Remove when done: sudo tc qdisc del dev eth0 root

Observe & Record:

→ How does measured latency compare to injected delay?
→ At what delay do clients start timing out?
→ What would you change in the code to handle high-latency networks?

Discussion Questions

Answer individually after completing the experiments.

22. What is the difference between a server that has crashed and a server that is simply very slow? Which is easier to handle in a distributed system, and why?

Answer:

23. When Node 2 was partitioned with iptables DROP, the client hung for several seconds before timing out. Where does this timeout come from? What part of your code controls it?

Answer:

24. The server handles each client connection in a separate goroutine. What are the limits of this approach? What would happen if 10,000 clients connected simultaneously?

Answer:

25. In Experiment D, why is DROP different from REJECT at the network level? Which better simulates a real network partition?

Answer:

Results Recording

Experiment	Avg Latency	Throughput	Success Rate	Key Observation
A — Baseline				
B — Overload				
C — Server Crash				
D — Partition DROP				
D — Partition REJECT				
E — 200ms latency				
E — 500ms latency				
E — 1000ms latency				

Go Quick Reference

Use these during the lab when filling in the TODO sections.

Network

```
// Connect to server with timeout
conn, err := net.DialTimeout("tcp", "10.8.100.22:7000", 5*time.Second)
if err != nil { log.Printf("connect failed: %v", err); return }
defer conn.Close()

// Set deadline before reading
conn.SetReadDeadline(time.Now().Add(5 * time.Second))

// Set deadline before writing
conn.SetWriteDeadline(time.Now().Add(5 * time.Second))

// Read a line
reader := bufio.NewReader(conn)
line, err := reader.ReadString('\n')

// Write to connection
fmt.Fprintf(conn, "REQUEST %d-%d\n", workerID, requestID)
```

Concurrency

```
// Launch a goroutine
go func(id int) {
    defer wg.Done()      // signal completion
    doWork(id)
}(workerID)

// WaitGroup — wait for all goroutines
var wg sync.WaitGroup
wg.Add(1)                // before launching goroutine
wg.Wait()                // blocks until all Done() called

// Atomic counter — thread-safe, no mutex needed
var counter atomic.Int64
counter.Add(1)           // increment
counter.Load()           // read value

// Range over channel until closed
for result := range results {
    // process result
}
```

Error Handling

```
// Go returns errors as values — always check them
conn, err := net.DialTimeout("tcp", addr, timeout)
if err != nil {
    log.Printf("Error: %v", err)
    return
}

// Common network errors you will see in experiments:
// connection refused  → server not running
// i/o timeout         → read/write deadline exceeded
```

```
// connection reset    → server crashed mid-connection
```

Important Rules

⛔ **Rules for This Lab**

1. Only use YOUR group's containers — container names contain your group number. 2. Never run docker stop or docker rm — this affects other groups. 3. Never touch k8s_ containers on 10.8.100.2 — they are system containers. 4. If your environment breaks completely — tell your instructor. They will reset your group. 5. Always kill the server with pkill server_bin before recompiling.

Port Reference

Group	Port	Group	Port	Group	Port	Group	Port
1	7000	6	7005	11	7010	16	7015
2	7001	7	7006	12	7011	17	7016
3	7002	8	7007	13	7012	18	7017
4	7003	9	7008	14	7013	19	7018
5	7004	10	7009	15	7014	20	7019

📌 **Next Lab**

Lab 02 will build a real distributed hash table (Chord DHT) across all 5 nodes, using the same Go networking foundation you learned here.

Student usernames	group1 – group20
Student passwords	group1@123 – group20@123
Lab ports	7000 – 7019 (Group 1=7000, Group 2=7001, … Group 20=7019)